TECHNICAL EXPERIMENT REPORT

OrderFlow: Distributed Cache Architectural Evaluation

Campaign 2026-10-11 15:46 UTC | smoke profile | 150 arrivals/s | 25 s per load test | 2 app replicas

Executive decision

ADOPT WITH CONDITIONS. Use a shared Redis cache for the expensive read model, with post-commit invalidation on mutable data. Add request coalescing and cache-outage admission controls. Do not select application-local L1 caches or sharding based on this campaign. Transactional inventory reservations remain database-authoritative.

- **Normal operation:** warm cache-aside delivered 150.0 RPS and 0.621 ms p95; the DB-only saturated at 82.8 completed RPS and 8,947 ms p95.
- **Freshness:** basic aside returned stale data 60/60 times; write invalidation 0/60; L1/L2 29/60.
- **Stampede protection:** the single flush trial favored protection (p95 down 47.0%, p99 down 64.7%).
- **Cache outage:** 100.4 completed RPS, 6.50 s p95, 492 dropped + 342 interrupted; 0% HTTP errors covers completed requests only.
- **Evidence quality:** one short smoke campaign, incomplete E01-E19 coverage, no per-second time series or replications.

Architectural decision matrix

Candidate	Performance evidence	Correctness/failure evidence	Decision
DB-only	82.8 RPS; 8.95 s p95	Overloaded without cache	Reject as read path
Basic cache-aside	150 RPS; 0.62 ms warm p95	60/60 stale after write	Reject as-is
Invalidate on write	Outage: 100.4 RPS	0/60 stale, but poor fallback	Adopt conditionally
Protected reads	0.73 ms p95 after flush	No outage/consistency trial	Retest combined mode
L1 + L2	0.75 ms p95; no benefit	29/60 stale, replica divergence	Defer
Cache sharding	Not measured	Not measured	No decision

Scope warning: Observed latency differences compare a saturated uncached system with a lightly loaded cache-hit regime. They demonstrate an architectural benefit under this workload, but are not a statistically established maximum-speedup claim.

1. Campaign design and evidence coverage

- Host: Linux-7.2.9-2-cachyos-x86_64-with-glibc2.44; 8 logical CPUs; CPU/memory quotas, disk, and competing activity not provided.
- 150 fixed scheduled arrivals/s x 25 seconds (~3,750 arrivals), 180 initial k6 VUs, max 700, 4-second graceful stop; two application replicas.
- Recorded initial dataset: 250,000 activity rows. TTL = 30 seconds, local L1 TTL = 1 second. DB delay = 0 ms.
- Seven k6 summaries: E01, E02, E03, E12a, E12b, E18 and E10. Three E17 consistency probes; not a full E01-E19 campaign.
- All load tests had 0% writes. Standard tests use 90% hot-key request fraction; the two flush tests use 100% hot-key requests.
- Raw request traces and time-aligned Prometheus series absent. Every figure is based on summary measurements.

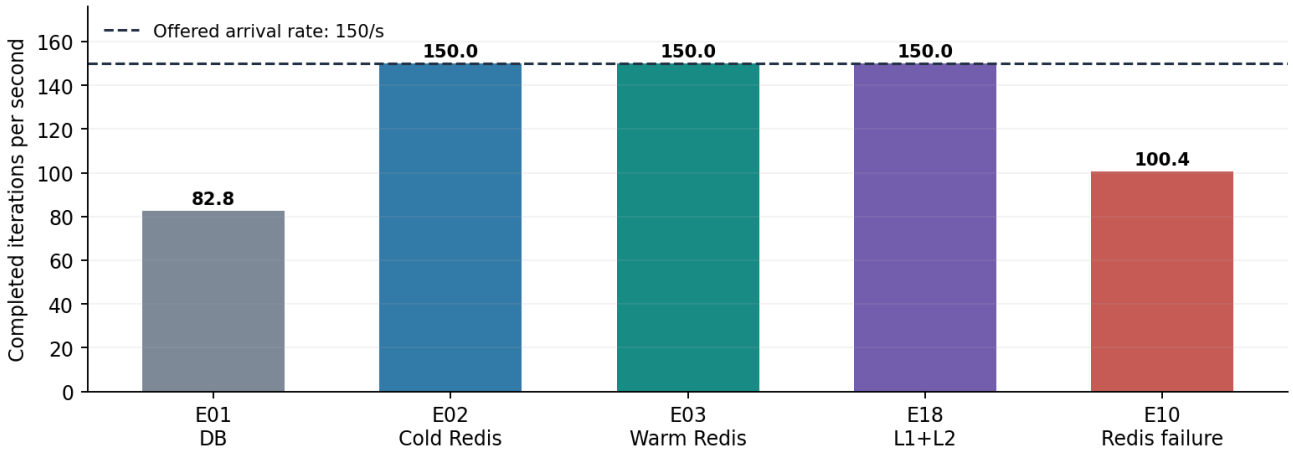


Figure 1. Completed iterations per second (k6 reporting rate). The 150/s dashed line is the offered arrival target.

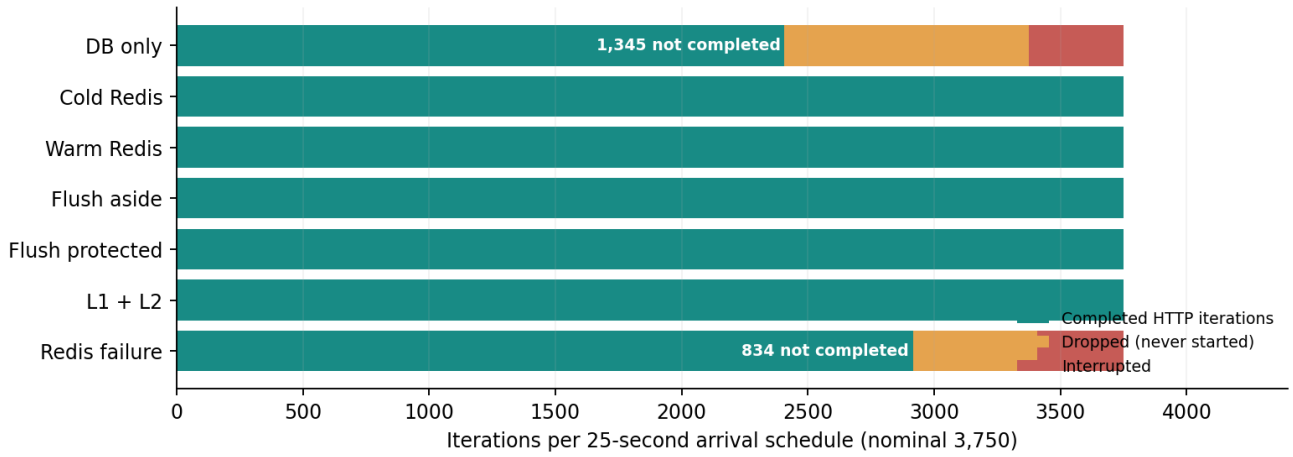


Figure 2. Completed HTTP iterations, scheduled arrivals dropped by k6, and interrupted iterations extracted from k6 log endings.

A zero-error response rate does not cover dropped or interrupted scheduled work. In E01, 968 arrivals were dropped and 377 interrupted; in E10, 492 were dropped and 342 interrupted. This is an availability/capacity problem despite a 0% reported HTTP failure rate.

2. Measured throughput and tail latency

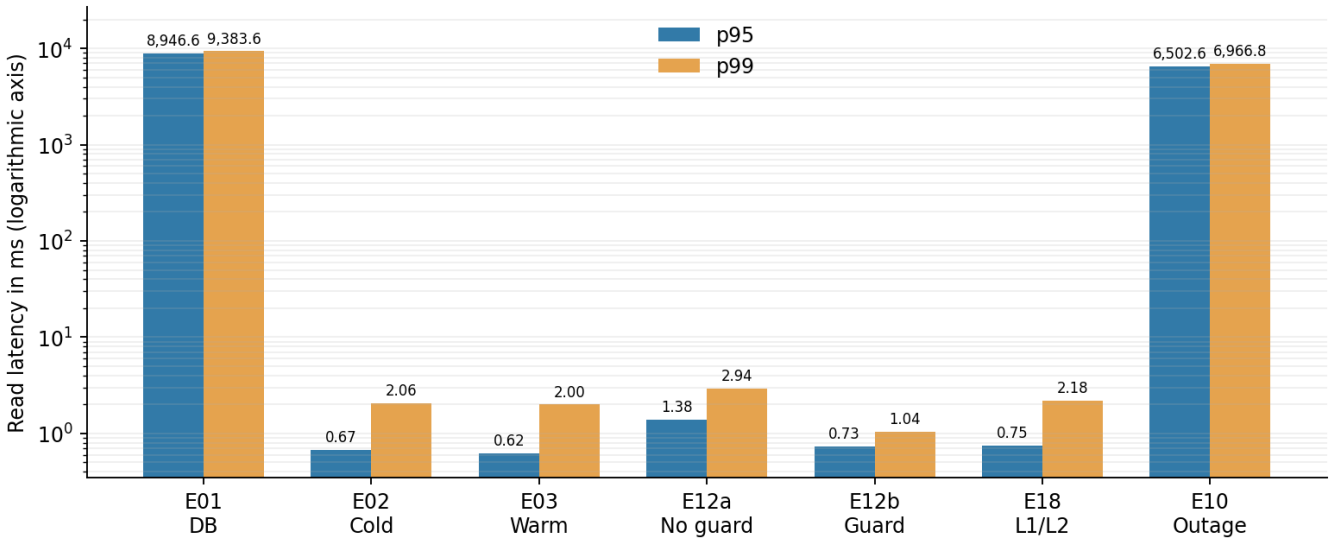


Figure 3. p95 and p99 of completed read requests. Logarithmic scale preserves sub-millisecond and multi-second measurements.

Run / mode	RPS	p95 ms	p99 ms	Hit %	Dropped	Interrupted
E01 / db	82.8	8,946.65	9,383.60	0.0	968	377
E02 / aside	150.0	0.67	2.06	97.1	0	0
E03 / aside	150.0	0.62	2.00	97.5	0	0
E12a / aside	150.0	1.38	2.94	99.7	0	0
E12b / protected	150.0	0.73	1.04	100.0	0	0
E18 / l1l2	150.0	0.75	2.18	97.5	0	0
E10 / invalidate	100.4	6,502.60	6,966.77	40.1	492	342

The warm-cache run completed 150.0 RPS against 150 offered, versus 82.8 RPS for DB-only (a 1.81x achieved-throughput ratio under a capped arrival rate). DB-only p95 was ~14,416 times warm-cache p95, but the two were in different saturation states. The cache capacity ceiling was **not** reached.

3. Cache locality and database relief

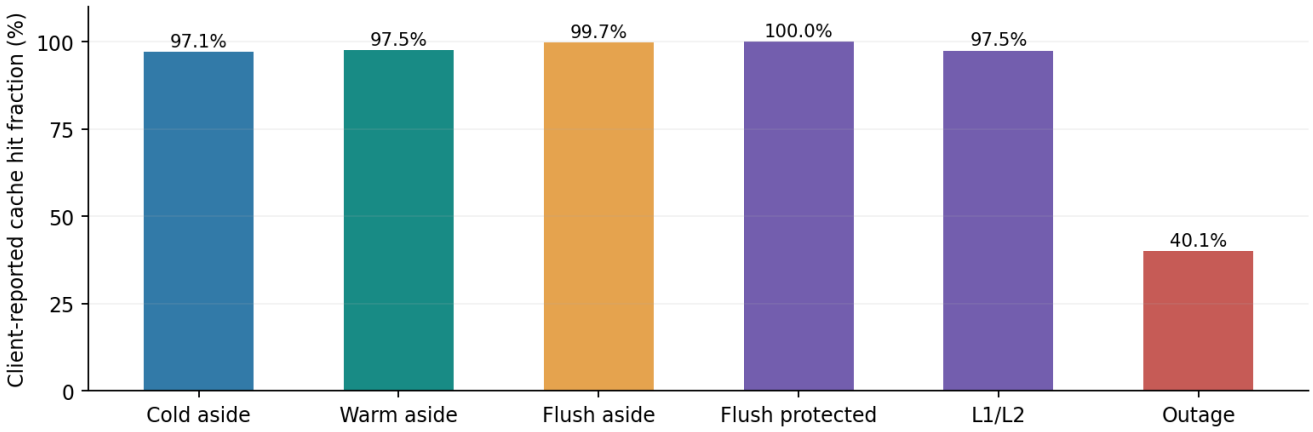


Figure 4. Fraction of completed client requests recorded as cache hits by each workload.

At 90% hot-key access locality, cold and warm cache-aside both achieved more than 97% hits over 25 seconds. The initially cold run warmed quickly; therefore E02 is not a controlled pure cache-miss test. The Redis-outage run had only a 40.1% hit fraction. E12a/b used 100% hot-key access and are a paired flush test rather than directly comparable to E03.

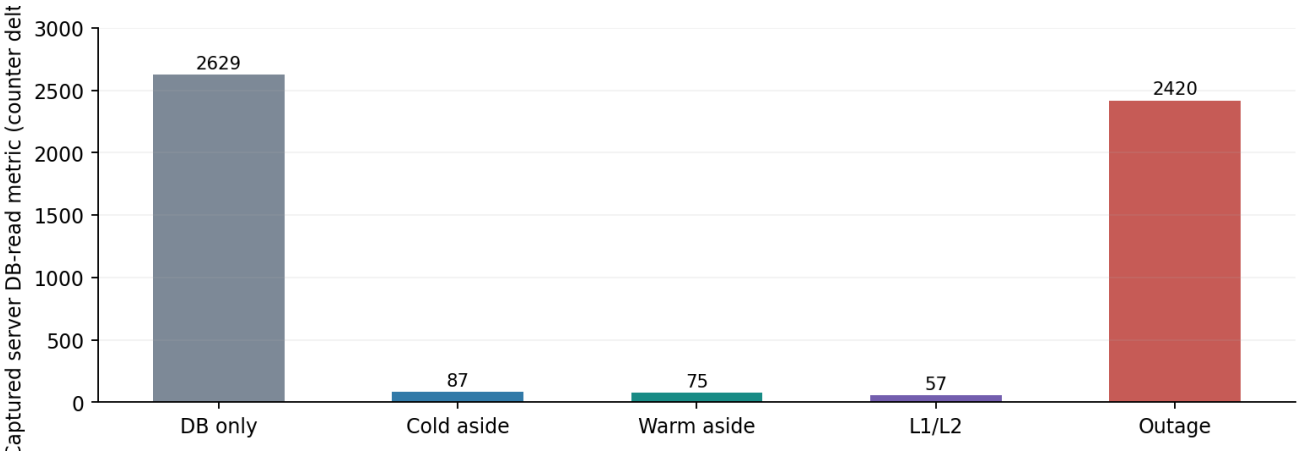


Figure 5. Captured non-integer server DB-read monitoring values. These are diagnostic counter deltas, not a reconciled query audit.

The monitored DB-read values suggest a strong decline with cache usage, but do not reconcile exactly with completed HTTP counts. For example, the protected-flush sample reports nonzero L2 misses but zero DB reads. Without raw counters and time windows, no precise DB-query savings percentage should be asserted.

4. Consistency is the decisive architectural trade-off

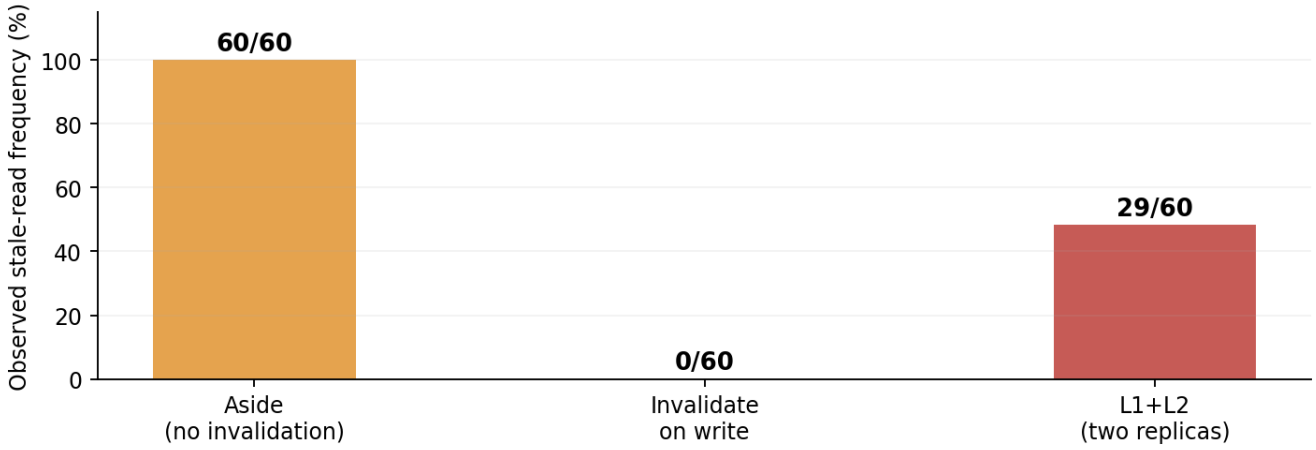


Figure 6. Stale reads during 60 immediate post-update probes per selected mode (E17).

Mode	Observed transition	Stale reads	Interpretation
Cache-aside	Rev 0 -> 1	60/60; 100%	No invalidation left an old projection in cache.
Invalidation	Rev 1 -> 2	0/60; 0%	All observed responses reported updated revision.
L1/L2	Rev 2 -> 3	29/60; 48.3%	App 1 showed rev 3; app 2 showed rev 2.

The observed L1/L2 divergence is especially important: updating a shared Redis entry does not automatically invalidate application-local caches on other replicas. Simple cache-aside also requires an explicit staleness contract. A result of 0/60 stale reads is a positive functional check, not a proof of linearizability under concurrent writes or failures.

Domain implication

Treat cached product and inventory displays as advisory **read projections**. A customer checkout/stock reservation must still execute atomic database operations and validate invariants against the source of truth. Cache invalidation should occur after a committed update (or via a durable outbox/change feed), not before commit.

5. Cache flush and stampede-protection experiment

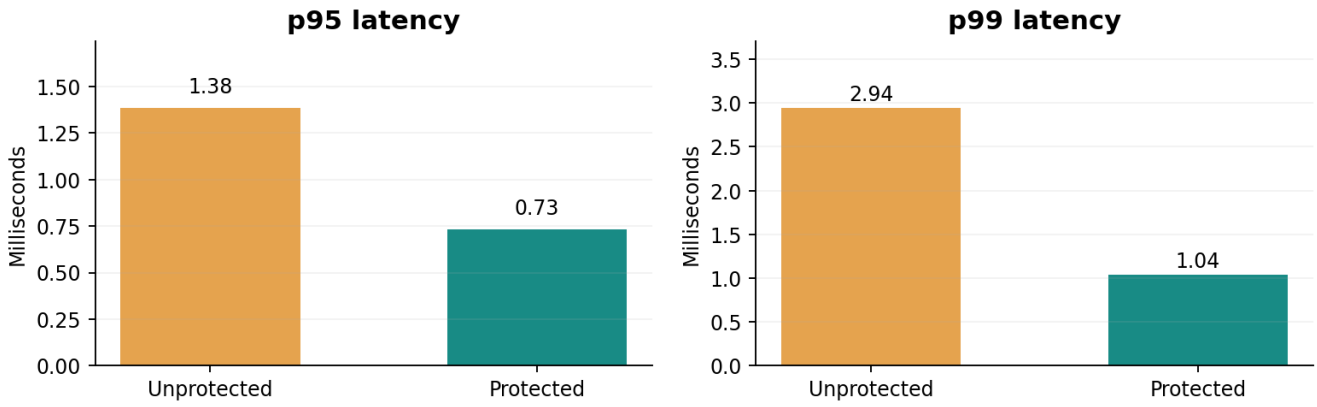


Figure 7. E12a vs E12b after a Redis flush under 100% hot-key traffic: one 25-second pair, 150 offered requests/s.

Measure	Unprotected E12a	Protected E12b	Observed change
Achieved RPS	150.03	150.03	No material change
Read p95	1.383 ms	0.733 ms	47.0% lower
Read p99	2.943 ms	1.040 ms	64.7% lower
Dropped / interrupted	0 / 0	0 / 0	Both met offered load
Client cache hits	99.68%	99.97%	+0.29 percentage points

The protected implementation improved p95 and p99 under the tested flush scenario. However, the report contains only campaign-wide k6 percentiles; there is no per-second latency/DB-QPS trace marking the actual flush event. The data do not establish that the gain persists at higher arrival rates, under diverse keys, or across repeated randomized runs.

Secondary finding: L1/L2 at normal load

Warm Redis-only E03 p95 = 0.621 ms; L1/L2 E18 p95 = 0.749 ms (20.7% higher). E18 p99 was also higher (2.178 vs 1.996 ms). At 150/s there is no demonstrated p95 advantage to offset the cross-replica staleness seen in E17. Stronger loads and a coordinated invalidation design would be necessary to reconsider it.

6. Redis outage: graceful fallback is not capacity resilience

E10 stopped Redis during load under the invalidate mode. Requests that completed did not report HTTP errors, indicating a functioning fallback for some requests. But the load generator could not keep up with offered demand.

Indicator	Warm cached E03	Redis outage E10
Completed throughput	150.0 RPS	100.4 RPS
HTTP p95	0.621 ms	6,502.6 ms
HTTP p99	1.996 ms	6,966.8 ms
Client hit rate	97.5%	40.1%
Completed scheduled iterations	3,751	2,916
Dropped arrivals	0	492
Interrupted iterations	0	342
Errors among completed	0%	0%

The outage reduced achieved throughput by approximately 33.0% relative to the no-fault warm cache run (strategies differ, so this is descriptive). E10 served ~77.8% of its 3,750 offered arrivals within the test and graceful stop window, while the remaining 834 were dropped or interrupted. p95 rose to 6.5 seconds. This is not an acceptable failure-mode performance result even though every completed HTTP response passed k6 checks.

Resilience remediation

- Implement Redis timeout budgets and a circuit breaker (fail fast), rather than piling up slow cache retries.
- Bound PostgreSQL fallback concurrency and per-key recomputation; provide rate limiting/load shedding under DB saturation.
- Use last-known-good stale reads only for noncritical read projections, labeled with freshness age; never for checkout reservations.
- After redis restart, use selective warming, jittered TTL and request coalescing to prevent cold-cache stampede.
- Retest with failure-event-aligned 1-second metrics and track recovery time, including the completion rate of scheduled requests.

7. Hypothesis decisions and experiment validity

Hypothesis	Disposition	Reason
H1. Cache lowers p95 >50%	Supported for this load	9s vs 0.62ms, but queueing regimes differ.
H2. Throughput at least 2x	Undetermined	Observed 1.81x achieved, cached rate capped at 150/s.
H3. Hits reduce DB load	Directional support	~97% client hits; DB counter collection not auditable.
H4. Survive Redis outage	Not satisfied	6.5s p95, 834 iterations not completed.
H5. Protection limits stampede	Tentative support	Single flush pair improves p95/p99, no aligned timeseries.
H6. L1/L2 beneficial	Not supported	No p95 benefit and 48.3% targeted stale reads.

Threats to validity

- **Single run, no variability bounds.** Differences have no confidence intervals or significance tests.
- **Partial campaign.** Missing TTL sweeps, hit-rate targets, repeated outage and restart, DB slowdown, concurrency/replicas scale tests, write-heavy workloads, and sharding.
- **Short duration.** 25-second phases cannot capture long-tail steady state, memory pressure, TTL lifecycle, or recovery after prolonged incidents.
- **Host and environment.** Single Linux host, 8 logical CPUs, unknown resource quotas and concurrent load.
- **Monitoring gaps.** k6 aggregates rather than per-second history; Prometheus-derived DB counters are not fully reconciled.
- **Workload bias.** High hotspot fraction and intentionally expensive database aggregation favor read caches; generality to normal DB endpoints is unknown.
- **Fault comparability.** E10 uses invalidate, not E03 aside; E12 uses 100% hot-key, not E03 90%; E17 uses sequential revisions.

8. Final conclusion and follow-up acceptance gates

Decision: ADOPT WITH CONDITIONS. In the measured flash-sale projection, shared Redis caching is compelling: it allowed all 150 offered requests/s to complete at sub-millisecond p95 under normal conditions, whereas the DB-only path saturated badly. However, basic caching is not safe for mutable order/inventory projections without invalidation, and Redis failure created an overload state. These issues outweigh any unsupported claim that simply enabling Redis makes the complete architecture scalable and resilient.

Recommended target architecture: Rust application replicas -> shared Redis read cache, invalidated after committed changes, protected by per-key coalescing and bounded staleness where business-permitted -> PostgreSQL authoritative transactional state. Enforce database admission controls and cache failure circuit breakers. Defer L1/L2 and sharding until measured need and consistent cross-replica invalidation are demonstrated.

Mandatory tests before final production ADR

- **P0 / failure gate:** repeat Redis stop/restart with protected invalidation, bounded fallback, and recovery SLO; count completed/offered arrivals and latency.
- **P0 / consistency gate:** concurrent writes/reads, target both replicas, TTL races and cache reconnection; guarantee database stock correctness.
- **P1 / sustainable capacity:** step 25/50/100/150/250+ RPS until first SLO breach; 3-5 independent repetitions per alternative.
- **P1 / stampede:** repeat randomized flush pair; capture p99, DB QPS and re-computation concurrency by second.
- **P2 / alternatives:** evaluate TTL, 5/20/50% writes, 1/2/4/8 replicas, L1 invalidation, then sharding if Redis saturates.
- **Measurement:** save timestamped samples, exact DB-query deltas, container saturation, versions and experiment manifest.

Data provenance

This report was produced from the supplied campaign ZIP: campaign.json, manifest.json, comparison.csv, seven k6 JSON summaries and logs, and three E17 consistency JSON results. All chart data are observed campaign aggregates. No results were simulated and no new benchmarks were executed.

Interpretation boundary: This is strong classroom evidence of performance/consistency/resilience trade-offs, but it is not sufficient for a production acceptance claim or a technology benchmark independent of workload.